StagePass Live
Arquitectura hexagonal

TALLER 13 · ARQUITECTURA DE SOFTWARE · UNAB

Diseño e implementación de una plataforma de reservas y tickets para conciertos, teatro y festivales.

La solución separa las decisiones del negocio de la interfaz web, el almacenamiento y los proveedores externos. El núcleo se ejecuta y prueba sin bases de datos ni credenciales comerciales.

Entrega preparada

Artefacto	Contenido
Aplicación web	Reserva, cotización, confirmación asíncrona simulada, descarga y verificación de tickets.
Código fuente	Entidades, puertos, casos de uso, composición y adaptadores.
UML formal	10 vistas UML; archivo Draw.io editable y PDF vectorial.
Validación	64 pruebas automatizadas aprobadas y recorrido de compra en navegador.

Alcance de los adaptadores

HMAC-SHA256 y generación SVG están implementados completamente. MySQL, MongoDB, PayPal, Wompi, PSE, WhatsApp, SendGrid y DIAN se representan con mocks funcionales, conforme a la opción permitida por el enunciado. No hay cobros, mensajes ni facturas reales.

Docente del enunciado: MSc. Feisar Moreno. Fuente del caso: “13 - Taller Arquitectura Hexagonal.pdf”, suministrado por el usuario.

01. Arquitectura y dependencias

La frontera esencial está entre el núcleo y los mecanismos externos. BookingService coordina los casos de uso y depende de contratos que expresan funciones del negocio. Los adaptadores implementan esos contratos y traducen los formatos tecnológicos.

Zona	Responsabilidad / archivos
Dominio	core/models.py: entidades inmutables, estados, cálculo de tarifas y vigencia.
Puertos	core/ports.py: BookingUseCases y contratos secundarios de catálogo, órdenes, pagos, tickets y comunicaciones.
Aplicación	core/service.py: reserva, confirmación, expiración, consulta, tickets y despacho de efectos.
Entrada	adapters/http.py y scheduler.py: solicitudes web, webhooks y reloj periódico.
Salida	adapters/storage.py, payments.py, tickets.py y communications.py.
Composición	bootstrap.py construye adaptadores y los inyecta en BookingService.

Inversión de dependencias

HTTP → BookingUseCases ← BookingService → PaymentGateway ← WompiMock. Las flechas de dependencia del código apuntan al contrato, aunque el flujo de ejecución alcance un servicio externo. Reemplazar una pasarela no exige cambiar las reglas de tarifas ni de inventario.

El núcleo no importa HTTP, JSON, SQL, MongoDB, frameworks, criptografía ni archivos. Sus imports se verifican con una prueba de inspección del árbol de sintaxis. Los puertos reciben entidades o valores del dominio, no requests ni DTO de SDK.

Modelo funcional

Un catálogo legado aporta la descripción de cada asiento. Su estado dinámico se deriva de las órdenes de una función específica. Así, dos conciertos que comparten recinto no comparten indebidamente sus bloqueos de asientos.

02. Modelo de dominio y UML

Enunciado	Clase del código	Datos y comportamiento
Evento / Función	Event	id, name, kind, venue_id, starts_at, state. Tipos: Concierto, Teatro y Festival.
Localidad / Asiento	Seat	id, venue_id, location, base_price, state. Estado por función, no global al recinto.
Cliente	Client	id, name, email, mobile, nationality; validate().
Reserva / Orden	Order	Cliente, asientos, cotización, creación, vencimiento, estado, pasarela y referencia. holds_at(), expire().
Valores monetarios	Quote	subtotal, fee, discount, total; calculate(seats).
Ticket	Ticket	id, order_id, code, security_hash, issued_at y claims.
Evidencia firmada	TicketClaims	Instantánea de datos inmutables de compra y ticket.
Entrega pendiente	Delivery	Clave idempotente, orden, canal, completed y attempts.

Lectura del anexo UML

Las diez vistas cubren dominio, contrato de entrada, adaptadores primarios, almacenamiento, pagos, tickets, comunicaciones, valores auxiliares, evidencia firmada y mapa de dependencias. Las vistas detalladas presentan atributos y operaciones; los nombres corresponden al código.

Asociación: línea continua. Composición: rombo lleno en el propietario. Dependencia: línea discontinua con flecha abierta. Realización: línea discontinua con triángulo vacío hacia la interfaz. Herencia: línea continua con triángulo vacío.

Archivo formal editable: docs/StagePassLive_UML.drawio. Versión de lectura: docs/StagePassLive_UML.pdf. Abrir el archivo editable con Draw.io / diagrams.net; se conservan cajas, conectores, etiquetas y páginas.

03. Tarifas y límite de entradas

Sea B la suma de los precios base de los asientos. Tanto el cargo administrativo como el descuento se calculan sobre B. Esta interpretación se declara porque el enunciado no establece un orden de aplicación compuesto.

Cantidad	Cargo	Descuento	Total
1 a 2	10% de B	0% de B	B × 1,10
3 a 5	8% de B	5% de B	B × 1,03
6	5% de B	10% de B	B × 0,95

El tramo “más de cinco” queda limitado a seis entradas por la regla de máximo acumulado. La tasa fija del último tramo es un porcentaje de 5%, no un cargo monetario constante.

Ejemplo: entradas de COP 150.000

Entradas	Subtotal	Cargo	Descuento	Total COP
1	150.000	15.000	0	165.000
2	300.000	30.000	0	330.000
3	450.000	36.000	22.500	463.500
5	750.000	60.000	37.500	772.500
6	900.000	45.000	90.000	855.000

Control acumulado

El servicio suma entradas aprobadas y reservas pendientes todavía vigentes del mismo cliente y evento. Antes de crear otra orden exige acumulado + nueva cantidad ≤ 6. Incluir las reservas activas evita que varias órdenes simultáneas eludan el límite.

Órdenes rechazadas y expiradas no consumen cupo. El identificador del cliente es estable dentro de la demostración; en producción debe proceder de una identidad autenticada. No se usa correo o nombre como clave de control.

Los importes usan Decimal y redondeo ROUND_HALF_UP a dos decimales. El navegador solicita la cotización al núcleo y solo muestra el resultado.

04. Reserva, expiración y concurrencia

Paso	Operación
1. Validar	Cliente, evento programado y futuro, pasarela admitida, selección única de 1 a 6 asientos pertenecientes al recinto.
2. Serializar	Abrir transacción de OrderStore; revisar vencimientos y leer disponibilidad vigente.
3. Comprobar	Rechazar asientos bloqueados/vendidos y exceso del máximo acumulado.
4. Calcular	Crear la cotización y fijar created_at y expires_at = created_at + 10 minutos.
5. Registrar	Obtener referencia de pago del adaptador y guardar la orden pendiente. Un error impide guardar el bloqueo.
6. Expirar	Si now ≥ expires_at y sigue pendiente, pasar a Expirada y liberar disponibilidad y cupo.

La frontera temporal

Una confirmación aceptada a los 599,999999 segundos todavía puede aprobar. A los 600 segundos ya no puede emitir tickets. No se extiende el plazo al recibir un estado pendiente ni al recargar la página.

ExpiryScheduler solicita expire() cada segundo. Además, las operaciones de lectura y escritura revisan el reloj. Por ello el vencimiento lógico es exacto; la actualización material del estado puede ocurrir en el siguiente ciclo si nadie consulta.

Garantía de la demostración

MongoOrderStoreMock usa RLock para que comprobación, cupo y escritura sean indivisibles frente a otros hilos. Un snapshot permite revertir la transacción si falla una operación. Las entidades son inmutables, por lo que el snapshot no comparte objetos mutables del dominio.

La prueba de 20 solicitudes simultáneas por un asiento obtiene un solo ganador. Ocho intentos paralelos del mismo cliente consiguen como máximo seis reservas. Esta garantía cubre un proceso; un despliegue distribuido requiere control transaccional real e índices/condiciones de unicidad por función y asiento.

05. Pagos y emisión segura

El puerto PaymentGateway ofrece register(PaymentRequest) y verify(PaymentReference). Los tres adaptadores traducen sus referencias técnicas a ese contrato. El núcleo recibe VerifiedPayment y vuelve a validar coherencia con su orden.

Verificación	Motivo
Autenticidad	El adaptador comprueba la prueba de la notificación contra una transacción registrada.
Correspondencia	La referencia, orden, pasarela, moneda COP e importe deben coincidir.
Estado y tiempo	Solo approved y una orden pendiente aún vigente autorizan emisión.
Idempotencia	Una orden ya aprobada devuelve el resultado existente; no crea tickets adicionales.

Estados de la orden

Pendiente de Pago → Aprobada: confirmación válida y oportuna. Pendiente de Pago → Rechazada: rechazo verificado. Pendiente de Pago → Expirada: vencimiento. Los tres estados finales no retornan a pendiente en esta solución.

Una aprobación tardía o recibida para una orden ya rechazada no emite tickets y marca reconciliation=True. Esa marca permite detectar una obligación de conciliación; el mock no devuelve dinero ni afirma haber hecho un reembolso.

Atomicidad de emisión

La aprobación, todos los tickets y las tareas de salida se guardan dentro de una transacción. Si falla la firma del segundo ticket, no queda el primero emitido ni la orden aprobada. La confirmación puede reintentarse sin duplicados.

Notificación y factura

Se crean tres tareas: WhatsApp, SendGrid y DIAN. dispatch() intenta cada una usando una clave estable por orden/canal. Una falla queda pendiente para reintento y no invalida los tickets. Las salidas simuladas guardan el resultado una sola vez por clave.

06. Dos adaptadores completos

HmacTicketSigner

La firma usa HMAC-SHA256 con una clave aleatoria de al menos 32 bytes para cada evento. Los datos se serializan de manera canónica: claves ordenadas, separadores constantes y UTF-8. El secreto no se incorpora al ticket ni se envía al navegador.

Los claims incluyen identificadores de ticket, orden, evento, cliente y asiento, precio base, total de orden, fecha de creación y código de canje. Alterar un asiento o usar otro secreto produce una verificación falsa. compare_digest evita la comparación ordinaria de cadenas para validar firmas.

Las claves se crean al iniciar y se mantienen solo en memoria, al igual que las órdenes de esta demo. Una versión persistente debe almacenar y versionar secretos de forma segura para seguir verificando tickets después de reiniciar.

SvgTicketRenderer

Genera un SVG vectorial descargable de 960 × 420 con nombre y fecha del evento, localidad, código alfanumérico, orden y hash. Escapa los textos insertados para que un nombre con caracteres de XML no introduzca etiquetas ejecutables ni rompa el documento.

El código de canje contiene 128 bits aleatorios en representación hexadecimal. El gráfico indica explícitamente que se trata de una demostración académica sin validez comercial. No se afirma que incluya un código QR, pues el enunciado exige representación gráfica y seguridad, no un formato de código específico.

Qué acredita la firma

La verificación acredita integridad y autenticidad de los claims bajo la clave del evento. No cifra información ni detecta por sí misma un segundo uso. Un caso de uso de canje único exigiría registrar un estado de redención de forma atómica; no se presenta como funcionalidad implementada.

Pruebas de los adaptadores

Firma correcta, claim alterado, secreto de otro valor, clave demasiado corta, longitud del hash, código alfanumérico, SVG parseable y escape de etiquetas. Ambos adaptadores ejecutan su trabajo real, sin servicios externos.

07. Ejecución y validación

Iniciar en Windows

Extraer el ZIP; abrir una terminal en StagePassLive y ejecutar **python app.py**. También puede abrir iniciar.bat. Requiere Python 3.10 o superior. No necesita instalar paquetes. Visitar **http://127.0.0.1:8083**.

Elegir evento y asientos, completar datos ficticios y reservar. El botón de simulación prepara el resultado del proveedor y la interfaz lo entrega al endpoint de webhook. Tras aprobar se muestran los tickets y su verificación.

Resultados comprobados

Comprobación	Resultado
Pruebas automatizadas	64 aprobadas: negocio, adaptadores, transacciones, concurrencia y HTTP.
Separación arquitectónica	Imports del núcleo limitados a tipos estándar y módulos del propio núcleo.
Compra en navegador	Tres asientos VIP, total COP 463.500; contador 10:00; orden aprobada y tres tickets.
Firma visible	La interfaz muestra: “Firma válida: el ticket conserva su integridad”.
Artefactos	UML editable, anexo vectorial, informe, código e instrucciones de ejecución.

Comando reproducible de pruebas: **python -m unittest discover -s tests -v**. Registro completo: docs/pruebas.txt. Los casos temporales usan un reloj manual, sin alterar el reloj del sistema ni esperar diez minutos.

Límites de la entrega

El servidor es local y no implementa autenticación productiva. Los mocks usan memoria y se reinician al cerrar. No se han integrado cuentas ni bases de datos reales. Para producción faltan autorización, persistencia, disponibilidad distribuida, conectores oficiales, gestión duradera de secretos, conciliación operativa y políticas de reintento con backoff.

08. Trazabilidad y referencias

Requisito del taller	Evidencia entregada
Dominio sin infraestructura	core/models.py, ports.py y service.py; prueba de imports.
Entidades exigidas	Event, Seat, Client, Order y Ticket; UML 01 y 09.
10 minutos / liberación	Order.expire(), BookingService.expire(), ExpiryScheduler; pruebas de frontera.
Tarifas / máximo seis	Quote.calculate(), BookingService.reserve(); ejemplos y pruebas por tramo.
Pago exitoso / hash	BookingService.confirm(), HmacTicketSigner y TicketClaims.
Entrada web y webhook	Handler, interfaz web y POST /api/webhooks/{provider}.
MySQL / MongoDB	Catalog y OrderStore con sus mocks explícitos; UML 04.
Tres pasarelas	PaymentGateway y PayPalMock, WompiMock, PseMock; UML 05.
Tickets / mensajes / DIAN	SVG y HMAC completos; WhatsAppMock, SendGridMock y DianInvoiceMock.
UML formal e implementación	Draw.io de 10 páginas, PDF vectorial y proyecto ejecutable.

Fuentes

1. UNAB. *Taller Arquitectura Hexagonal*, caso StagePass Live. Enunciado suministrado; MSc. Feisar Moreno.
2. Alistair Cockburn. Hexagonal Architecture, artículo original. Referencia conceptual para fronteras y puertos.
3. Python Software Foundation. hmac — Keyed-Hashing for Message Authentication. Uso de HMAC y compare_digest.
4. Draw.io. Save a diagram in various formats. Formato nativo editable .drawio.